

UNIVERSIDAD MAYOR DE SAN ANDRÉS
FACULTAD DE CIENCIAS PURAS Y NATURALES
CARRERA DE INFORMÁTICA



PRACTICA INDIVIDUAL

Universitario:

-Charca Condori Ronaldo

RU. 1855786

Materia: Desarrollo de Aplicaciones Multimedia INF-263

DOCENTE: Ph.D. Moises Martin Silva Choque

AUXILIAR: Univ. Elizabeth Suzaño Condori

2026

LINK de Github

https://github.com/TAREASDEINFORMATICA/proyecto_individual_ronaldo_charca_condori

1. Introducción

El presente documento describe la instalación, funcionamiento y desarrollo de los productos individuales realizados para la materia Desarrollo de Aplicaciones Multimedia INF-263. El trabajo se divide en tres módulos: clasificación de texturas en imágenes, aplicación de un filtro de suavizado mediante una ventana de 3 x 3 píxeles y producción multimedia de un cover animado en Unity exportado para ejecución en navegador mediante WebGL.

La documentación fue organizada para que cualquier usuario pueda descargar los archivos desde GitHub, ejecutar los programas y comprender la lógica principal implementada en cada ejercicio. También se incluyen capturas de evidencia, fragmentos de código relevantes y recomendaciones para mejorar la presentación final del proyecto.

Objetivo general

Desarrollar y documentar tres aplicaciones multimedia que integren procesamiento básico de imágenes, generación de audio, animación 3D y publicación web mediante WebGL.

Objetivos específicos

- Implementar una aplicación capaz de diferenciar regiones de una imagen según colores asociados a césped, agua, tierra, asfalto y cemento.
- Aplicar un filtro de suavizado basado en el promedio de vecinos dentro de una matriz de 3 x 3 píxeles.
- Crear una escena en Unity con personajes animados, audio sincronizado y exportación en formato WebGL.
- Documentar el proceso de instalación y uso de cada entregable para facilitar su evaluación.

a) Clasificación de Texturas

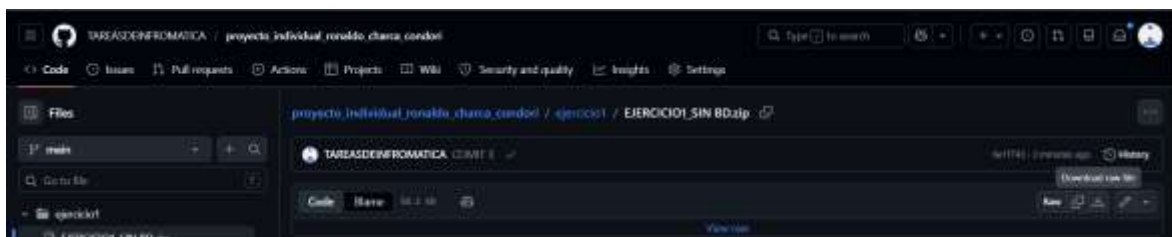
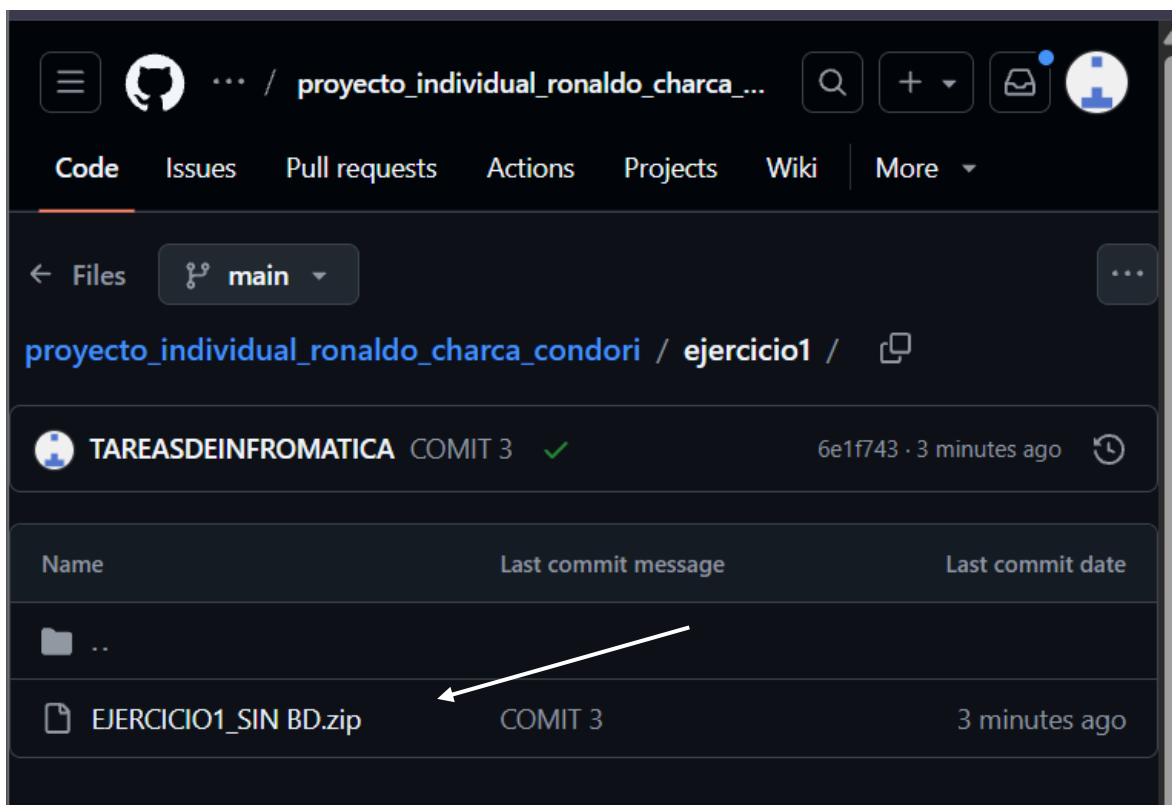
Desarrollar una aplicación capaz de diferenciar superficies dentro de una imagen, por ejemplo: césped, tierra, cemento o asfalto, aplicando principios similares a los utilizados en clasificación de imágenes satelitales.

Descripción del módulo.

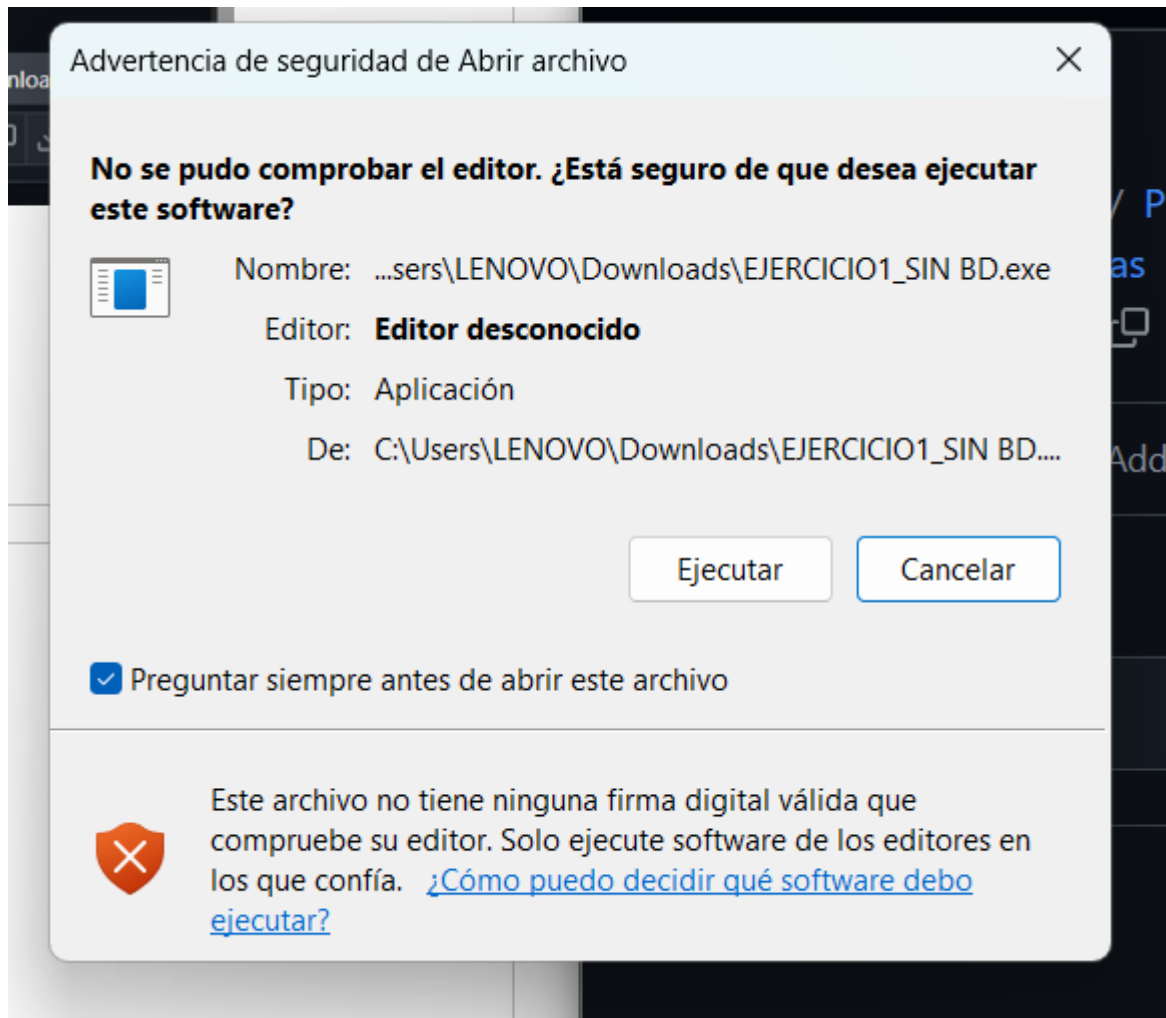
Este módulo consiste en una aplicación de escritorio desarrollada en C# con Windows Forms. Su propósito es cargar una imagen y clasificar visualmente algunas superficies según el color predominante de sus píxeles. El enfoque aplicado es similar a una clasificación básica por umbrales RGB, donde cada píxel se evalúa y se reemplaza por un color representativo de la clase detectada.

INSTALACION.

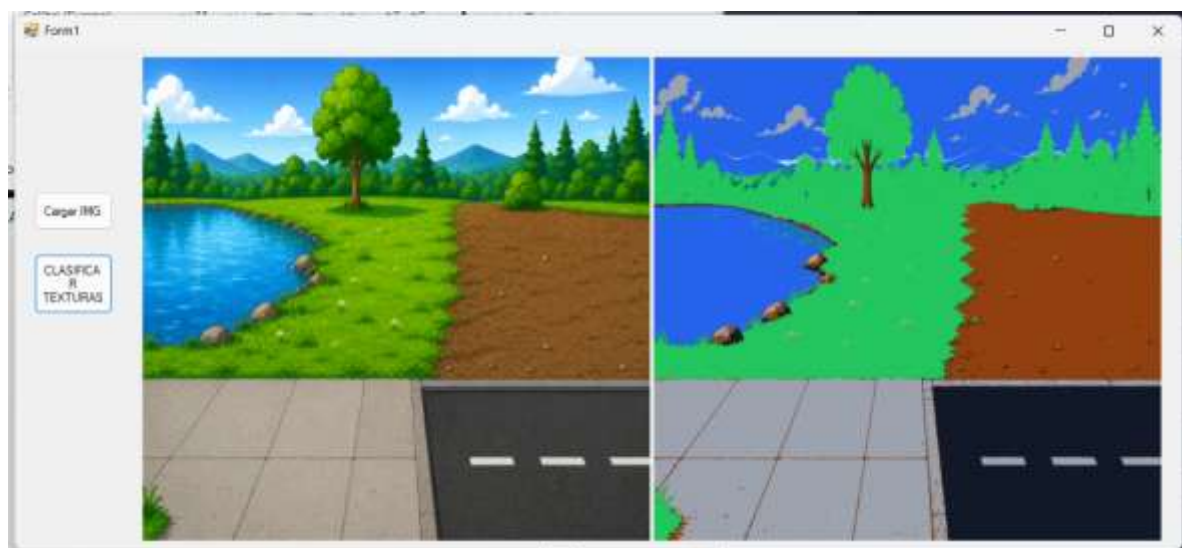
Descargamos de Github el archivo ejecutable .exe



Luego lo podemos ejecutar.

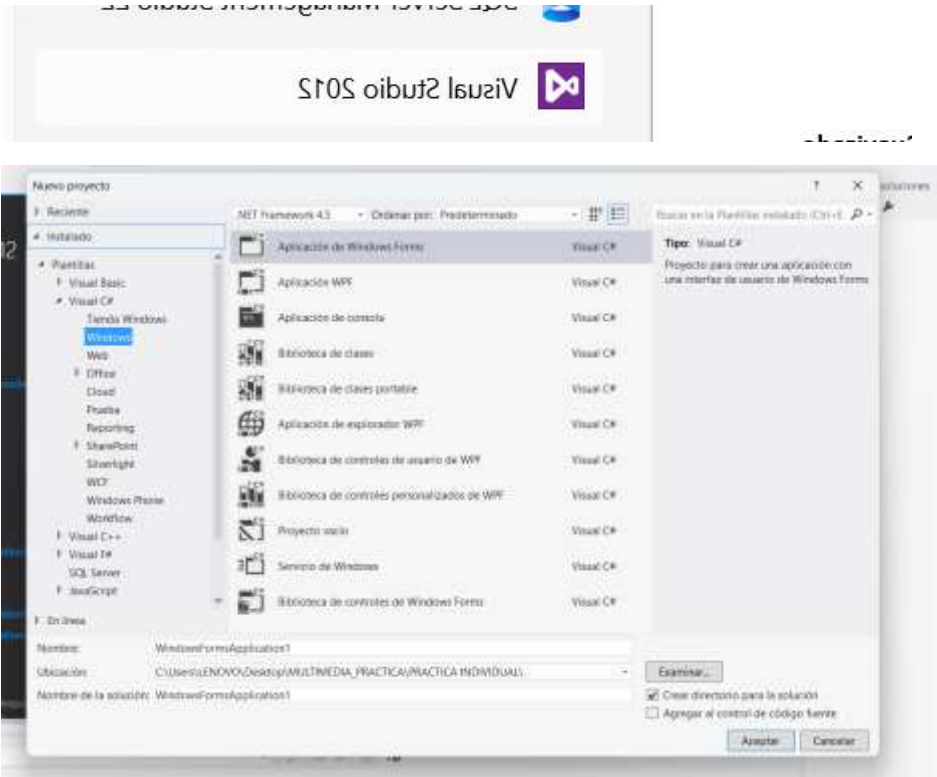


Luego selecciones una imagen para poder clasificar el suelo

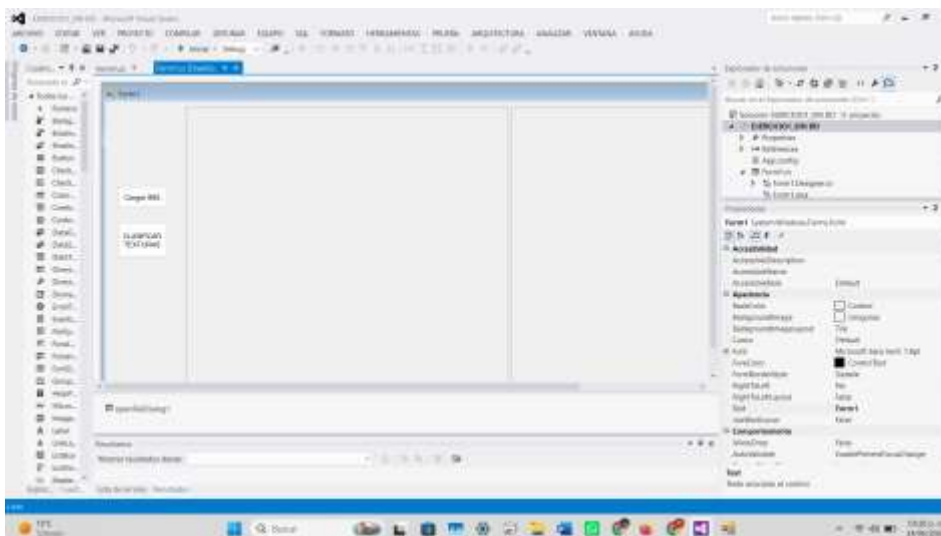


DOCUMENTACION.

Primero creamos un proyecto en Visual estudio 2012



La interfaz fue construida con dos PictureBox, dos botones y un OpenFileDialog. El primer PictureBox muestra la imagen original cargada por el usuario, mientras que el segundo muestra la imagen procesada. Los botones permiten cargar la imagen y aplicar la clasificación.



El algoritmo recorre la imagen píxel por píxel. Para cada punto obtiene los valores R, G y B, calcula un promedio de intensidad y aplica reglas condicionales para decidir si el color pertenece a césped, agua, tierra, asfalto o cemento. Finalmente, se construye una nueva imagen con colores representativos para cada textura.

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace EJERCICIO1_SIN_BD
{
    public partial class Form1 : Form
    {
        Bitmap bmp;
        Bitmap bpm2;

        public Form1()
        {
            InitializeComponent();
        }

        private void btnCargar_Click(object sender, EventArgs e)
        {
            OpenFileDialog abrir = new OpenFileDialog();
            abrir.Filter = "Imagenes|*.jpg;*.jpeg;*.png;*.bmp";
            if (abrir.ShowDialog() == DialogResult.OK)
            {
                bmp = new Bitmap(abrir.FileName);
                pictureOriginal.Image = bmp;
            }
        }

        private void btnClasificar_Click(object sender, EventArgs e)
        {
            if (bmp == null)
            {
                MessageBox.Show("Primero cargue una imagen.");
                return;
            }

            bpm2 = new Bitmap(bmp.Width, bmp.Height);

            for (int y = 0; y < bmp.Height; y++)
            {
                for (int x = 0; x < bmp.Width; x++)
                {
                    Color pixel = bmp.GetPixel(x, y);

                    int r = pixel.R;
                    int g = pixel.G;
                    int b = pixel.B;

                    int promedio = (r + g + b) / 3;

                    Color nuevoColor;

                    if (g > r + 20 && g > b + 20)
                    {
                        nuevoColor = Color.FromArgb(34, 197, 94); // Cesped
                    }
                }
            }
        }
    }
}
```

```

else if (b > r + 25 && b > g + 15)
{
    nuevoColor = Color.FromArgb(37, 99, 235); // Agua
}
else if (r > 90 && g > 45 && g < 160 && b < 120 && r > b)
{
    nuevoColor = Color.FromArgb(146, 64, 14); // Tierra
}
else if (promedio < 90 && Math.Abs(r - g) < 30 && Math.Abs(g - b) < 30)
{
    nuevoColor = Color.FromArgb(17, 24, 39); // Asfalto
}
else if (promedio >= 90 && Math.Abs(r - g) < 35 && Math.Abs(g - b) < 35)
{
    nuevoColor = Color.FromArgb(156, 163, 175); // Cemento
}
else
{
    nuevoColor = pixel;
}

bpm2.SetPixel(x, y, nuevoColor);
}
}

pictureResultado.Image = bpm2;
}

}
}

```

Luego lo probamos

Primero cargamos una img



Luego presionamos el botón clasificamos las texturas



Y se puede ver las texturas clasificadas .

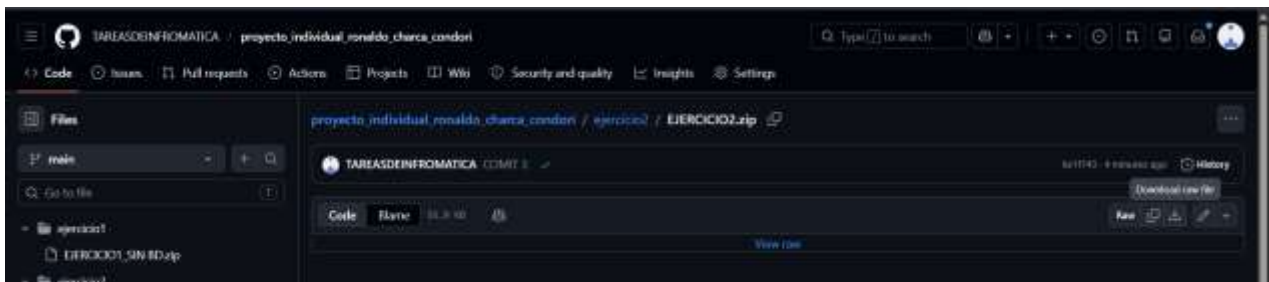
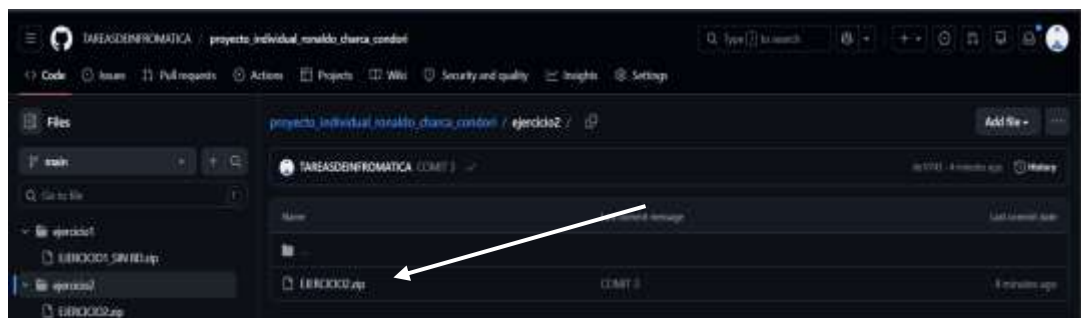
b) Implementación de un Filtro de Suavizado

Diseñar un filtro de promedio que recorra la imagen utilizando ventanas de 3x3 píxeles para reducir ruido y suavizar transiciones de color.

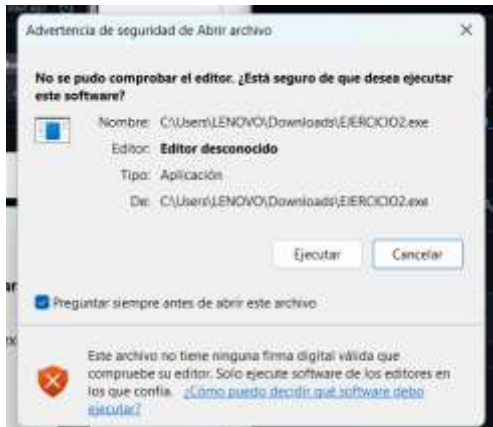
El segundo módulo implementa un filtro de promedio que recorre la imagen utilizando una ventana de 3 x 3 píxeles. El objetivo es reducir ruido visual y suavizar transiciones de color. La nueva imagen se obtiene calculando el promedio de los valores RGB de los nueve píxeles vecinos.

INSTALACION.

Descargamos de Github el archivo ejecutable .exe



Luego lo podemos ejecutar.

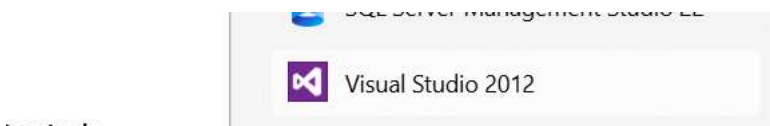


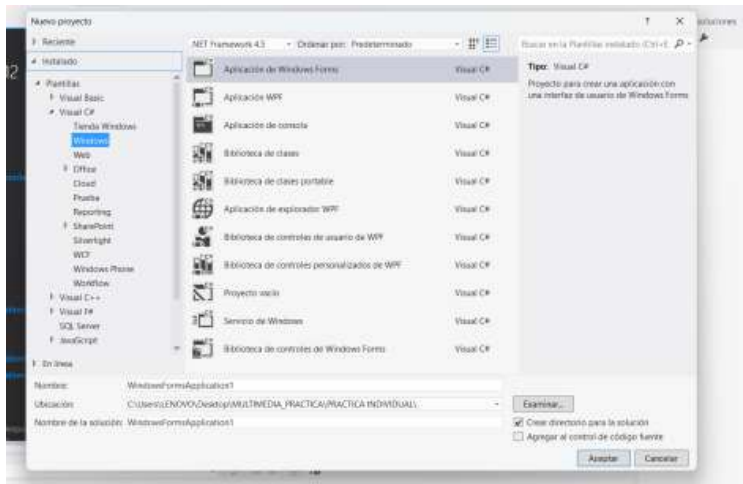
Luego selecciones una imagen para poder el suavizado



DOCUMENTACION.

Primero creamos un proyecto en Visual estudio 2012

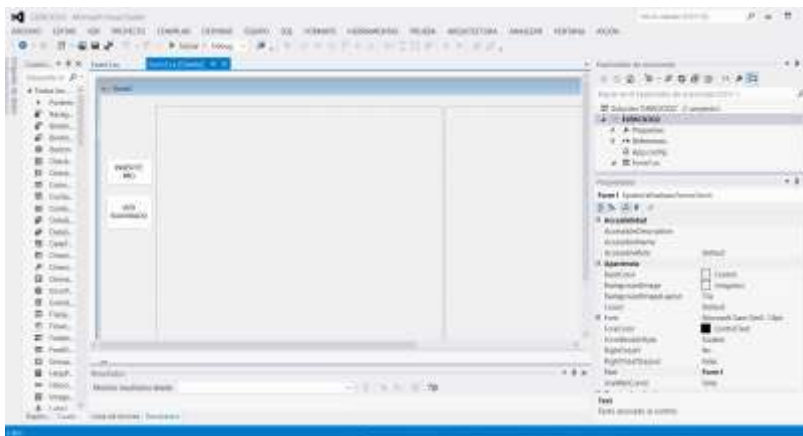




Luego en la pantalla creamos dos

picture box dos buttons y un openFileDialog

La interfaz conserva el mismo enfoque del primer ejercicio: dos PictureBox para comparar la imagen original y la imagen procesada, dos botones para cargar y procesar, y un OpenFileDialog para seleccionar archivos desde el equipo.



Para cada píxel interno de la imagen se suman los valores de rojo, verde y azul de sus ocho vecinos y de sí mismo. Luego cada suma se divide entre nueve para obtener el color promedio. En el proyecto, el filtro se aplica varias veces para conseguir un suavizado más visible.

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
```

```
namespace EJERCICIO2
{
    public partial class Form1 : Form
    {
        Bitmap imagenOriginal;
```

```

Bitmap imagenSuavizada;
public Form1()
{
    InitializeComponent();
}

private void button1_Click(object sender, EventArgs e)
{
    OpenFileDialog img = new OpenFileDialog();
    img.Filter = "Imagenes|*.jpg;*.png;*.bmp";

    if (img.ShowDialog() == DialogResult.OK)
    {
        imagenOriginal = new Bitmap(img.FileName);
        pictureBox1.Image = imagenOriginal;
    }
}

private void button2_Click(object sender, EventArgs e)
{
    if (imagenOriginal == null)
    {
        MessageBox.Show("Primero cargue una imagen.");
        return;
    }
    Bitmap temp = new Bitmap(imagenOriginal);

    temp = SuavizarImagen(temp);
    temp = SuavizarImagen(temp);
    temp = SuavizarImagen(temp);
    temp = SuavizarImagen(temp);

    imagenSuavizada = temp;
    pictureBox2.Image = imagenSuavizada;
}

private Bitmap SuavizarImagen(Bitmap original)
{
    Bitmap nueva = new Bitmap(original.Width, original.Height);

    for (int y = 1; y < original.Height - 1; y++)
    {
        for (int x = 1; x < original.Width - 1; x++)
        {
            int sumaR = 0;
            int sumaG = 0;
            int sumaB = 0;

            for (int j = -1; j <= 1; j++)
            {
                for (int i = -1; i <= 1; i++)
                {
                    Color pixel = original.GetPixel(x + i, y + j);

                    sumaR += pixel.R;
                    sumaG += pixel.G;

```

```

        sumaB += pixel.B;
    }
}

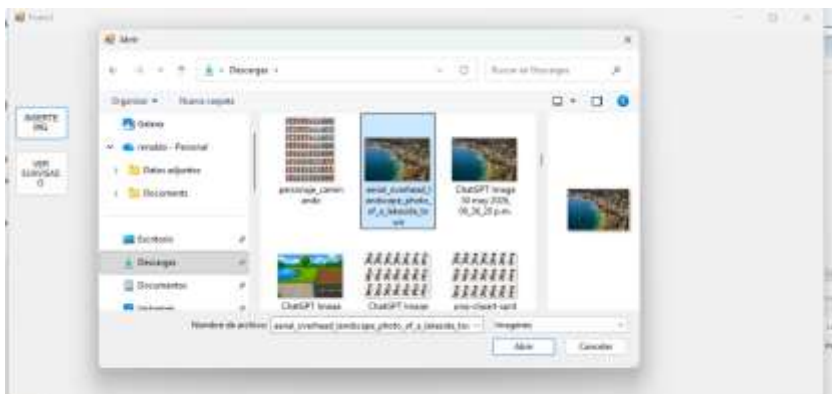
int r = sumaR / 9;
int g = sumaG / 9;
int b = sumaB / 9;

nueva.SetPixel(x, y, Color.FromArgb(r, g, b));
}
}
return nueva;
}
}
}

```

Luego lo probamos

Primero cargamos una img



Luego presionamos el botón de suavizado



Y se puede ver la imagen suabizada.

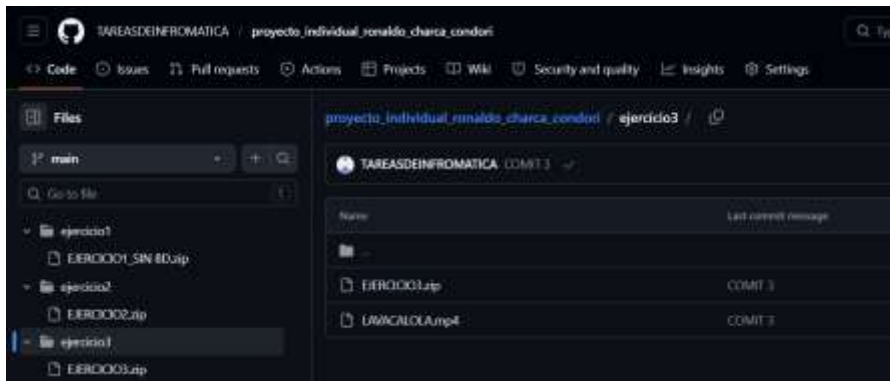
c) Cover de "La Vaca Lola"

El tercer producto multimedia consiste en una escena desarrollada en Unity 3D donde varios personajes realizan una coreografía sincronizada con una producción musical. El resultado puede visualizarse mediante un video o ejecutarse directamente en un navegador a través de una exportación WebGL.

Realizar una producción multimedia de la canción infantil utilizando la identidad de un artista al menos conocido, Vincular el audio con una animación (puede ser el modelo 3D o los monigotes) asegurando que el ritmo y el movimiento coincidan.

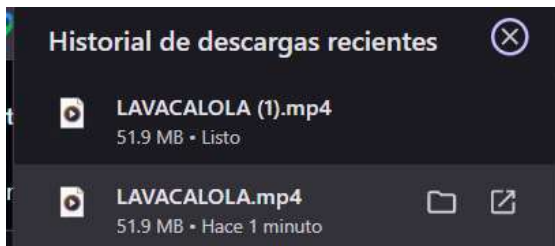
Podemos verlo por video o Web GL.

Primero descargamos de Github.



PODEMOS DESCARGAR CUALQUIERA DE LOS DOS.

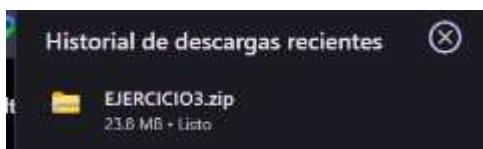
Primero el video



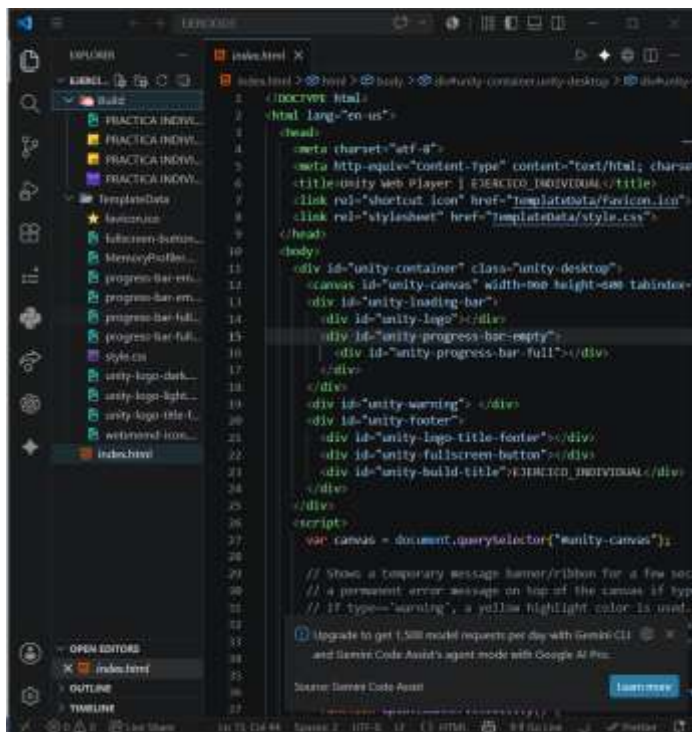
Lo podemos ver



La otra opciones es descargar el web gl

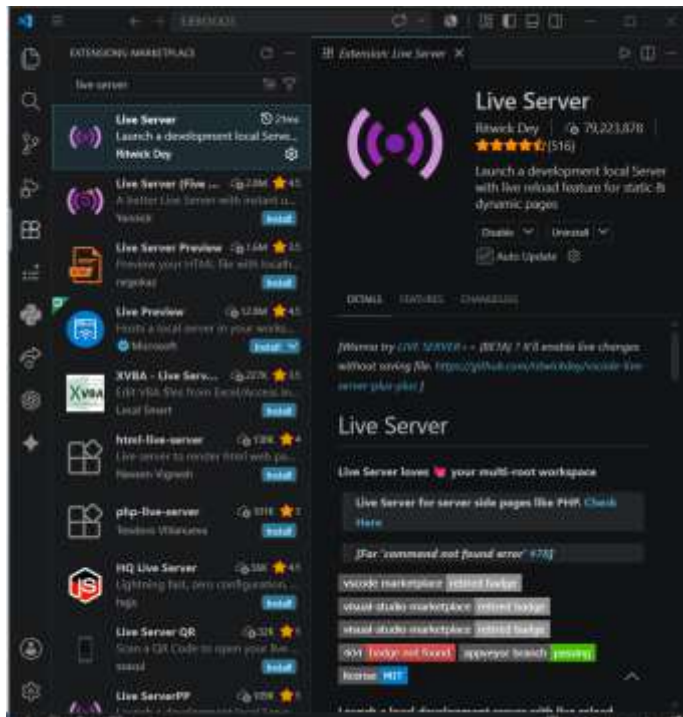


Primero lo descomprimos y lo abrimos con Visual estudio

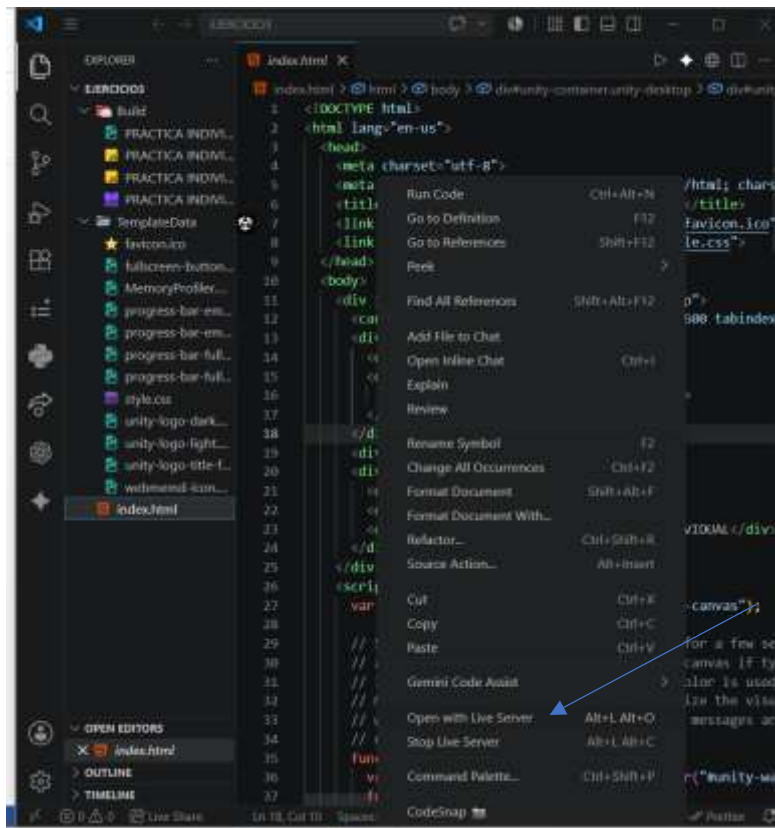


Luego le aemos run file pero si no quiero podemos usar Open live server

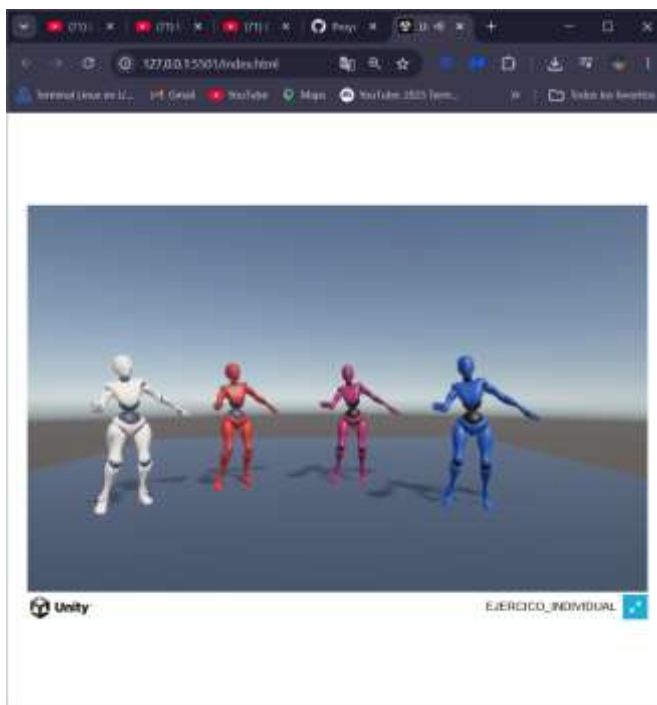
Descargamos open live server



Lo instalamos luego podemos ver que se puede ejecutar el html con el open live server.



Luego se ejecutara en su navegador predeterminado.



DOCUMENTACION.

Primero creamos un proyecto en Unity 3D

El desarrollo inicia con la creación de un proyecto 3D en Unity. Después se importan los personajes, animaciones y el audio final. Los personajes se colocan en la escena, se asignan sus animaciones y se sincroniza la música para que el movimiento coincida con el ritmo.



Luego descargamos la música que utilizaremos de

Primero la música hecha en colab con huggingface.co usamos Meta Platforms **MusicGen Small** para la música y para la voz cantada de un famoso usamos la ia de Alibaba Group **Qwen3-TTS-12Hz-1.7B-Base**.

Para una entrega académica responsable, se recomienda utilizar voces propias, voces sintéticas genéricas o audios de referencia autorizados, evitando clonar o imitar voces de artistas reales sin permiso.

Primero descargamos la musica

```
!pip install --upgrade pip
!pip install --upgrade transformers scipy accelerate
from google.colab import drive
drive.mount('/content/drive')
import os
import scipy.io.wavfile
from transformers import pipeline
carpeta = "/content/drive/MyDrive/multimedia"
os.makedirs(carpeta, exist_ok=True)
synthesiser = pipeline("text-to-audio", "facebook/musicgen-small", device=0)
prompt = ""
```

Música infantil alegre para una canción sobre una vaquita bailando.

Estilo reggae pop latino suave, divertido yailable para niños.

Ritmo tropical con percusión ligera, bajo reggae suave, palmas,

guitarra rítmica alegre, melodía simple y pegajosa.

Debe sonar como una base musical cantable para esta letra:

"La vaquita baila suave bajo el sol tropical,

moviendo la cola al ritmo reggae junto al palmar".

Ambiente feliz, tierno, juguetón y tropical.

Sin voces, solo música instrumental de fondo.

Duración aproximada de 30 segundos.

.....

```
music = synthesiser(prompt, forward_params={"do_sample": True, "max_new_tokens": 1500})
```

```
ruta_salida = carpeta + "/musica_fondo_30s.wav"
```

```
scipy.io.wavfile.write(
```

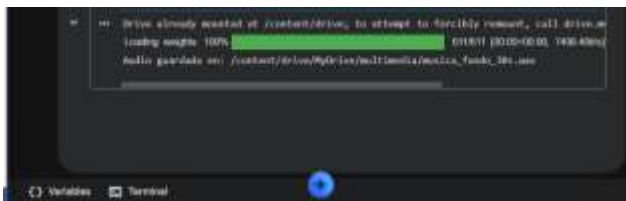
```
    ruta_salida,
```

```
    rate=music["sampling_rate"],
```

```
    data=music["audio"]
```

```
)
```

```
print("Audio guardado en:", ruta_salida)
```



 musica_fondo_30s.wav

9 jun

Luego la voz cantada para ello necesitamos darle una voz cantada de otra música y escribirle que es lo que dice la música el cantantes es MALULA.

```
# @title Instalar Dependencias
```

```
!pip install -U qwen-tts
```

```
!pip install -U flash-attn --no-build-isolation
```

```
!pip install -U accelerate
```

```
import qwen_tts
```

```
dir(qwen_tts)
```

```
import os
```

```
import torch
```

```
import soundfile as sf
```

```
from IPython.display import Audio, display
```

```
from qwen_tts import Qwen3TTSTModel
```

```
# REPRODUCIR AUDIO
```

```
def play_audio(file_path):
```

```

if os.path.exists(file_path):
    display(Audio(file_path))
else:
    print("Archivo no encontrado:", file_path)

# VERIFICAR GPU
device = "cuda" if torch.cuda.is_available() else "cpu"
print("Usando dispositivo:", device)

# CARGAR MODELO
base_model = Qwen3TTSTModel.from_pretrained(
    "Qwen/Qwen3-TTS-12Hz-1.7B-Base",
    device_map="cuda:0" if device == "cuda" else "cpu",
    dtype=torch.bfloat16 if device == "cuda" else torch.float32,
    attn_implementation="eager"
)
print("Modelo cargado correctamente")

# AUDIO DE REFERENCIA
ref_audio_path = "mluma.mp3"
ref_audio_text = ""

Hey que paso parceros, y si con otros pasas un rato
vamos a ser felis vamos a ser feliz felices los cuatro
y te agrandamos el cuerta y si con otro pasas el rato vamo a ser feliz
vamo a ser felices los cuatro y yo te asepto el trato y lo asemo
otro rato gracias por su cariño creo que estamos haciendo historia
no no creo estamos haciendo historia pero gracias a ustedes
los mejores fanaticos del mundo
los amo gracias por tanto amor por tanto cariño ehi se les quiere
y disfruten de mi nueva cancion felices los cuatro
bery boy bery boy beybi chau""

print("Clonando voz desde referencia...")

# CREAR PROMPT DE CLONACIÓN
voice_clone_prompt = base_model.create_voice_clone_prompt(
    ref_audio=ref_audio_path,
    ref_text=ref_audio_text,
    x_vector_only_mode=False)

# TEXTO NUEVO A GENERAR
target_text = ""

```

La vaquitaaaa baiiila suaaave bajo el sooooool tropical,
moviendo la cola al ritmo reggae junto al palmar.
Con sus paaaaasos pequeñiitos va marcando el compáaas,
mientras canta alegreeemente y no quiere paraaar.
La luuuna la miraaaa y las estreeellas también,
la vaquiita sigue bailando una y otra veeez.
Entre risaaas y canciones se escucha su voooz,
llenando de alegría todo el campo alrededor.
Baiiila, baiiila vaquiita, no dejes de soñaaaaar,
que la noooche está boniita y te invita a cantaaar.
Con el viiiiento en la pradera y las flores al pasaaaaar,
la vaquiita mueve el cuerpo y se pone a disfrutaaaar.
Su campaaana suena alegreee con un dulce tintineaaaar,
y los griiillos la acompañan con su ritmo natural.
En la plaaaaaya imaginariaaa, bajo un cielo de coloor,
la vaquiita cantaAAAA fueeerte con ternura y emoción.
Salta, giiira y se divieeerte hasta ver el amanecéeer,
porque cuando suena reggae ella vuelve a renaceeer.
Y al final de la canción, con cariño y mucha paaz,
la vaquiita se despide, pero vuelve a comenz aaaar.

.....

GENERAR AUDIO CLONADO

```
wavs, sr = base_model.generate_voice_clone(  
    text=target_text,  
    language="Spanish",  
    voice_clone_prompt=voice_clone_prompt)
```

GUARDAR RESULTADO

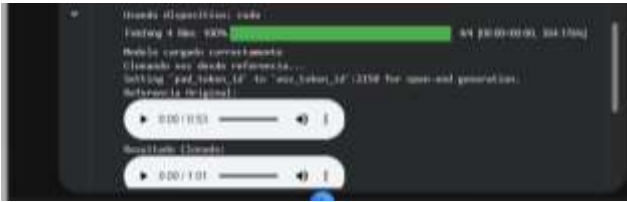
```
output_clone_path = "output_cloned2.wav"  
sf.write(output_clone_path, wavs[0], sr)
```

```
print("Referencia Original:")
```

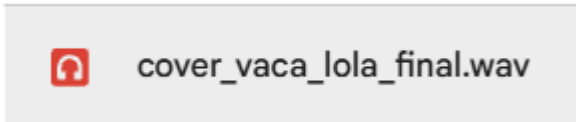
```
play_audio(ref_audio_path)
```

```
print("Resultado Clonado:")
```

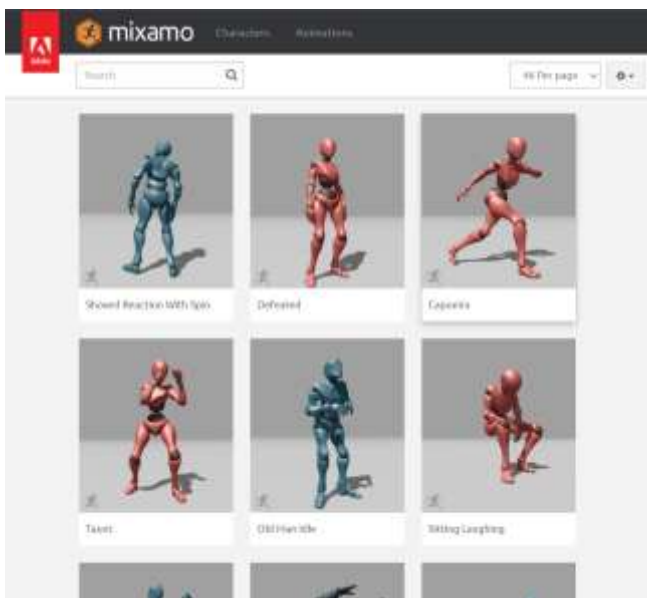
```
play_audio(output_clone_path)
```



Luego los juntamos los dos audios y tendremos la música escuchada en el video o el web gl.



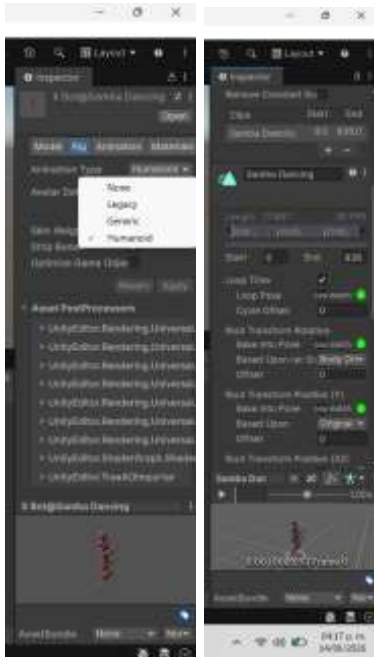
Luego descaramos el monigote en Mixamo



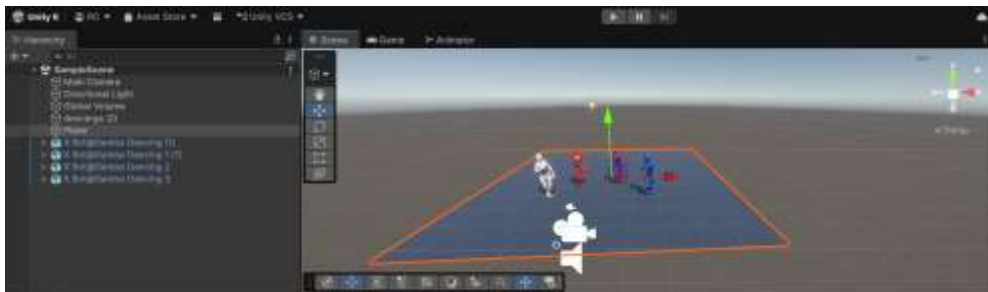
Luego exportamos los monigotes y la música al proyecto de unity



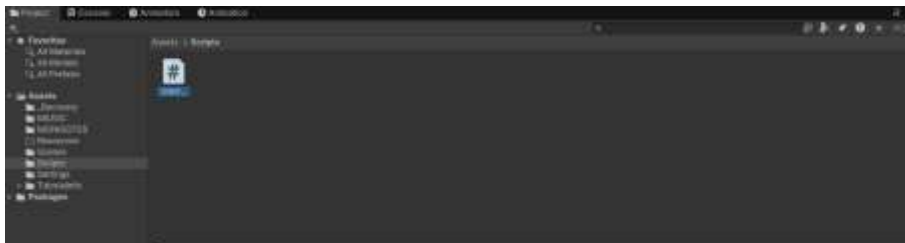
Donde le damos las propiedades a los monigotes



A cada onigote luego lo arastramos todos a SampleScene y la musica mas



Se creó un script para permitir el movimiento de la cámara con el teclado y la rotación con el mouse. Esto facilita revisar la coreografía desde diferentes ángulos durante la prueba de la escena.



using UnityEngine;

```
public class CONTROLAR_CAMARA : MonoBehaviour
{
    public float velocidad = 5.0f;
    public float velocidadRotacion = 100.0f;

    void Start()
    {
    }

    void Update()
    {
```

```

{
    // Movimiento con flechas o WASD
    float movimientoHorizontal = Input.GetAxis("Horizontal");
    float movimientoVertical = Input.GetAxis("Vertical");

    // Subir y bajar con E y Q
    float movimientoArribaAbajo = 0;

    if (Input.GetKey(KeyCode.E))
    {
        movimientoArribaAbajo = 1;
    }

    if (Input.GetKey(KeyCode.Q))
    {
        movimientoArribaAbajo = -1;
    }

    // Direccion de movimiento de la camara
    Vector3 direccion = new Vector3(
        movimientoHorizontal,
        movimientoArribaAbajo,
        movimientoVertical
    );

    // Mover la camara
    transform.Translate(direccion * velocidad * Time.deltaTime);

    // Rotar la camara con el mouse manteniendo clic derecho
    if (Input.GetMouseButton(1))
    {
        float rotacionHorizontal = Input.GetAxis("Mouse X");
        float rotacionVertical = Input.GetAxis("Mouse Y");

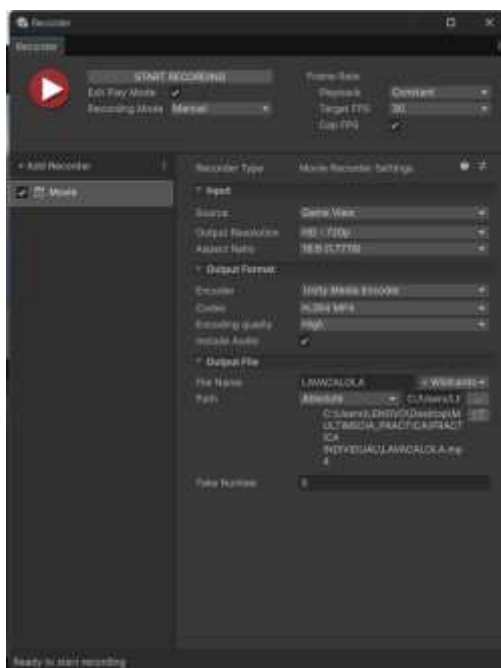
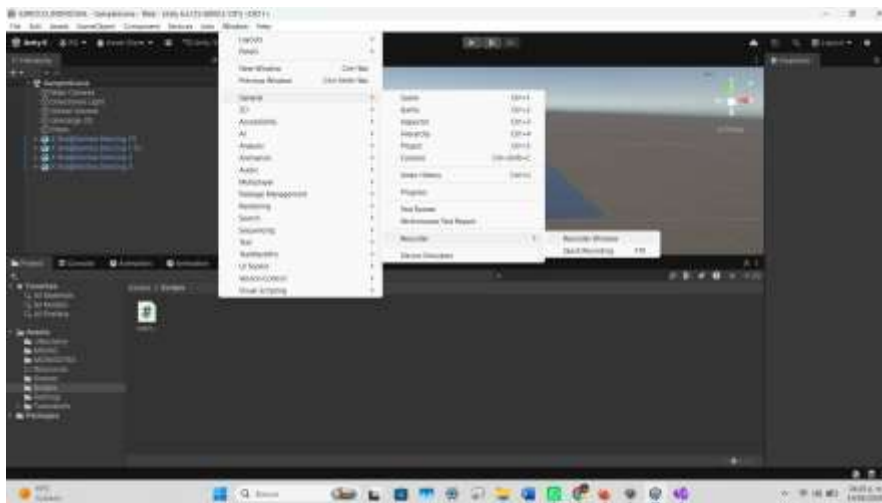
        transform.Rotate(Vector3.up, rotacionHorizontal * velocidadRotacion * Time.deltaTime, Space.World);
        transform.Rotate(Vector3.left, rotacionVertical * velocidadRotacion * Time.deltaTime, Space.Self);
    }
}
}

```

Luego por ultimo se puso un piso con material y podemos ejecutarlo.



Luego lo podemos grabar con



Sino lo exportación se realiza desde las opciones de Build de Unity, seleccionando la plataforma WebGL. Unity genera una carpeta con un archivo index.html y subcarpetas necesarias para la ejecución en navegador. Para evitar errores en servidores simples, se recomienda desactivar la compresión si el servidor no está configurado para entregar archivos comprimidos de Unity.

- Abrir File > Build Settings o Build Profiles.
- Seleccionar la plataforma WebGL y cambiar de plataforma si es necesario.
- Verificar en Player Settings que la compresión sea compatible con el hosting elegido.
- Ejecutar Build y elegir una carpeta de salida.
- Subir o ejecutar la carpeta generada mediante un servidor local o hosting compatible.

